BasicObject

The root of Ruby's class hierarchy, BasicObject, has only a handful of instance methods:

methods/basic_object.rb

```
im = BasicObject.instance_methods
im # => [:==, :equal?, :!, :!=, :instance_eval, :instance_exec, :__send__, :__id__]
```

If you don't specify a superclass, your classes inherit by default from Object, which is itself a subclass of BasicObject. If you want a *Blank Slate* (66), you can inherit directly from BasicObject instead. For example, if Computer inherited directly from BasicObject, then it wouldn't have a problematic display method.

Inheriting from BasicObject is the quicker way to define a Blank Slate in Ruby. In some cases, however, you might want to control exactly which methods to keep and which methods to remove from your class. Let's see how you can remove a specific method from a class.

Removing Methods

You can remove a method from a class by using either Module#undef_method or Module#remove_method. The drastic undef_method removes any method, including the inherited ones. The kinder remove_method removes the method from the receiver, but it leaves inherited methods alone. Let's look at a real-life library that uses undef_method to create a Blank Slate.

The Builder Example

The Builder gem is an XML generator with a twist. You can generate XML tags by calling methods on Builder::XmlMarkup:

methods/builder_example_1.rb

```
require 'builder'
xml = Builder::XmlMarkup.new(:target=>STDOUT, :indent=>2)

xml.coder {
  xml.name 'Matsumoto', :nickname => 'Matz'
  xml.language 'Ruby'
}
```

This code produces the following snippet of XML:

```
<coder>
  <name nickname="Matz">Matsumoto</name>
  <language>Ruby</language>
</coder>
```